4. Workflow Automation — Автоматизация рабочих процессов с AI

Фундаментальные принципы автоматизации рабочих процессов

1.1. Концепция Workflow Automation

Workflow Automation — это технология автоматизации бизнес-процессов и рабочих потоков, при которой задачи, данные и файлы передаются между участниками системы согласно заранее определённым правилам и последовательностям без ручного вмешательства

Эволюция автоматизации:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                        ЭВОЛЮЦИЯ АВТОМАТИЗАЦИИ                            │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ЭТАП 1: РУЧНАЯ АВТОМАТИЗАЦИЯ (1990-2005)                               │
│  ────────────────────────────────────────                                │
│  • Скрипты и макросы                                                     │
│  • Требует программирования                                              │
│  • Жёсткая логика, сложное изменение                                     │
│                                                                          │
│  ЭТАП 2: LOW-CODE ПЛАТФОРМЫ (2005-2018)                                 │
│  ───────────────────────────────────────                                 │
│  • Визуальные конструкторы                                               │
│  • Интеграция через API                                                  │
│  • Zapier, IFTTT, Microsoft Flow                                         │
│                                                                          │
│  ЭТАП 3: AI-ENHANCED AUTOMATION (2019-2024)                              │
│  ──────────────────────────────────────────────                          │
│  • Интеграция с LLM и AI-моделями                                        │
│  • Умная маршрутизация задач                                             │
│  • n8n, Make, ApiX-Drive                                                 │
│                                                                          │
│  ЭТАП 4: AUTONOMOUS WORKFLOWS (2025-2026)                                │
│  ────────────────────────────────────────                                │
│  • AI-агенты управляют workflows                                         │
│  • Динамическая адаптация процессов                                      │
│  • Self-healing workflows                                                │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

Математическая модель рабочего процесса:

```
Workflow = {T, C, R, S, E}

где:
  T = {t₁, t₂, ..., tₙ} — множество задач (tasks)
  C — условия перехода между задачами (conditions)
  R — правила бизнес-логики (rules)
  S — состояние workflow (state)
  E — события триггеры (events)
```

Функция выполнения workflow:

```
Execute(Workflow, Input) → Output

где:
  S₀ = Initial_State
  Sᵢ₊₁ = Transition(Sᵢ, tᵢ, Inputᵢ)
  Output = Final_State(Sₙ)
```

1.1.1. Ключевые компоненты Workflow Automation

Компонент	Описание	Пример
Trigger	Событие, запускающее workflow	Webhook, расписание, сообщение
Action	Конкретное действие в workflow	Отправка email, запись в БД
Condition	Логическое условие для ветвления	IF статус = "approved" THEN...
Loop	Циклическое выполнение действий	FOR EACH элемент в списке
Delay	Задержка между действиями	WAIT 5 minutes

Компонент	Описание	Пример
Error Handler	Обработка ошибок	ON ERROR → notify admin

1.1.2. Преимущества автоматизации workflows

Преимущество	Описание	Метрика улучшения
Скорость	Автоматическое выполнение без задержек	10-100x ускорение
Точность	Исключение человеческих ошибок	99.9% точность
Масштабируемость	Обработка растущего объёма задач	Линейное масштабирование
Аудит	Полное логирование всех действий	100% трассируемость
Интеграция	Связь разрозненных систем	Единая экосистема

1.2. Архитектура платформ автоматизации

1.2.1. Общая архитектура workflow-платформы

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    АРХИТЕКТУРА WORKFLOW ПЛАТФОРМЫ                        │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                     ПРЕЗЕНТАЦИОННЫЙ СЛОЙ                         │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐            │   │
│  │  │  Web UI      │  │  API         │  │  CLI         │            │   │
│  │  │  (Editor)    │  │  Gateway     │  │  Tools       │            │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘            │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                  │                                       │
│                                  ▼                                       │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                     СЛОЙ ОРКЕСТРАЦИИ                             │   │
│  │  ┌──────────────────────────────────────────────────────────┐    │   │
│  │  │  Workflow Engine                                         │    │   │
│  │  │  • Parser (чтение определений workflow)                  │    │   │
│  │  │  • Scheduler (планирование выполнения)                   │    │   │
│  │  │  • Executor (выполнение действий)                        │    │   │
│  │  │  • State Manager (управление состоянием)                 │    │   │
│  │  └──────────────────────────────────────────────────────────┘    │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                  │                                       │
│                                  ▼                                       │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                     СЛОЙ ИНТЕГРАЦИЙ                              │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │   │
│  │  │  HTTP/REST   │  │  Webhook     │  │  Database    │  │  Message     │  │   │
│  │  │  Connector   │  │  Handler     │  │  Connector   │  │  Queue       │  │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  │   │
│  │  │  AI/LLM      │  │  File        │  │  Auth        │  │  Custom      │  │   │
│  │  │  Connector   │  │  Storage     │  │  Provider    │  │  Code        │  │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                  │                                       │
│                                  ▼                                       │
│  ┌──────────────────────────────────────────────────────────────────┐   │
│  │                     СЛОЙ ДАННЫХ И МОНИТОРИНГА                    │   │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐            │   │
│  │  │  Execution   │  │  Audit       │  │  Metrics     │            │   │
│  │  │  Logs        │  │  Trail       │  │  Dashboard   │            │   │
│  │  └──────────────┘  └──────────────┘  └──────────────┘            │   │
│  └──────────────────────────────────────────────────────────────────┘   │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

1.2.2. Типы триггеров workflows

Таблица 1.1. Классификация триггеров

Тип триггера	Описание	Пример использования	Частота
Webhook	HTTP-запрос от внешней системы	Поступление заявки с сайта	On-demand
Schedule	По расписанию (cron)	Ежедневная выгрузка отчётов	Периодически
Polling	Опрос внешнего источника	Проверка новых писем	Каждые N минут
Event	Событие в системе	Изменение статуса заказа	On-change
Manual	Ручной запуск	Административные задачи	По требованию

Пример конфигурации триггера (JSON):

```json
{
  "trigger": {
    "type": "webhook",
    "name": "new_application",
    "path": "/api/webhook/application",
    "method": "POST",
    "authentication": {
      "type": "bearer_token",
      "token_env": "WEBHOOK_SECRET"
    },
    "filter": {
      "field": "status",
      "operator": "equals",
      "value": "new"
    }
  }
}
```

1.3. Обзор платформ автоматизации (2026)

1.3.1. Сравнительный анализ платформ

Таблица 1.2. Топ-6 платформ Workflow Automation

Платформа	Лицензия	Self-hosted	РФ API	Цена	Сложность	Рекомендация
n8n	Fair-code	✅ Да	✅ Через HTTP	Бесплатно*	Средняя	⭐⭐⭐⭐⭐
ApiX-Drive	Проприетарная	❌ Нет	✅ Native	От 990₽/мес	Низкая	⭐⭐⭐⭐⭐
Make	Проприетарная	❌ Нет	⚠️ Ограничено	От $9/мес	Средняя	⭐⭐⭐
Zapier	Проприетарная	❌ Нет	❌ Нет	От $20/мес	Низкая	⭐⭐
Node-RED	Apache 2.0	✅ Да	✅ Через HTTP	Бесплатно	Высокая	⭐⭐⭐⭐
Activepieces	MIT	✅ Да	✅ Через HTTP	Бесплатно	Средняя	⭐⭐⭐⭐

*n8n бесплатен для self-hosted использования

1.3.2. Детальный обзор n8n

n8n — это расширяемая платформа автоматизации рабочих процессов с fair-code лицензией, позволяющая создавать сложные workflows через визуальный редактор или код

Ключевые особенности n8n:

Особенность	Описание	Преимущество
Self-hosted	Можно развернуть на своём сервере	Контроль данных, 152-ФЗ
Node-based	Визуальный редактор на основе узлов	Интуитивное создание
JavaScript	Возможность кастомизации через код	Гибкость реализации
900+ интеграций	Готовые коннекторы к сервисам	Быстрое подключение
AI Nodes	Встроенная поддержка LLM	Интеграция с AI-агентами

Архитектура n8n:

```
┌──────────────────────────────────────────────────────────────────────────┐
│                              АРХИТЕКТУРА N8N                             │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
```

```
│  ┌──────────────────────────────────────────────────────────────┐
│  │  EDITOR (Веб-интерфейс)                                      │
│  │  • Drag-and-drop workflow editor                             │
│  │  • Node configuration panels                                 │
│  │  • Execution preview                                         │
│  └──────────────────────────────────────────────────────────────┘
│                                  │                               │
│                                  ▼                               │
│  ┌──────────────────────────────────────────────────────────────┐
│  │  WORKFLOW ENGINE                                             │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐      ||
│  │  │  Parser      │  │  Executor    │  │  Scheduler   │      ||
│  │  └──────────────┘  └──────────────┘  └──────────────┘      ||
│  └──────────────────────────────────────────────────────────────┘
│                                  │                               │
│                                  ▼                               │
│  ┌──────────────────────────────────────────────────────────────┐
│  │  NODES (Интеграции)                                          │
│  │  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ┌──────────────┐  ||
│  │  │  Trigger     │  │  Action      │  │  AI/LLM      │  │  Custom      │  ||
│  │  │  Nodes       │  │  Nodes       │  │  Nodes       │  │  Code        │  ||
│  │  └──────────────┘  └──────────────┘  └──────────────┘  └──────────────┘  ||
│  └──────────────────────────────────────────────────────────────┘
│                                  │                               │
│                                  ▼                               │
│  ┌──────────────────────────────────────────────────────────────┐
│  │  DATA LAYER                                                  │
│  │  • PostgreSQL/SQLite (хранение workflows)                    │
│  │  • Redis (кэширование, очереди)                              │
│  │  • Binary data storage (файлы)                               │
│  └──────────────────────────────────────────────────────────────┘
│                                                                  │
└──────────────────────────────────────────────────────────────────┘
```

1.3.3. Российские альтернативы

ApiX-Drive — ведущая российская платформа:

Параметр	Значение
Разработчик	Россия (г. Москва)
Год запуска	2017
Интеграции	400+ сервисов
152-ФЗ	✅ Полное соответствие
Оплата	✅ Карты РФ
Поддержка	✅ На русском языке
Цена	От 990₽/месяц

Преимущества российских платформ:

```
┌──────────────────────────────────────────────────────────────────┐
│                    ПРЕИМУЩЕСТВА РОССИЙСКИХ ПЛАТФОРМ              │
├──────────────────────────────────────────────────────────────────┤
│                                                                  │
│  ЮРИДИЧЕСКИЕ:                                                    │
│  ────────────────                                                │
│  • Соответствие 152-ФЗ о персональных данных                     │
│  • Хранение данных на территории РФ                              │
│  • Российская юрисдикция                                         │
│  • Работа с российскими сервисами (Яндекс, Сбер, VK)             │
│                                                                  │
│  ТЕХНИЧЕСКИЕ:                                                    │
│  ────────────────                                                │
│  • Локализация интерфейса (русский язык)                         │
│  • Поддержка российских часовых поясов                           │
│  • Интеграция с российскими API (Yandex, GigaChat)               │
│  • Техподдержка в рабочее время РФ                               │
│                                                                  │
│  ФИНАНСОВЫЕ:                                                     │
│  ────────────────                                                │
│  • Оплата картами российских банков                              │
│  • Счета для юридических лиц РФ                                  │
│  • Нет валютных ограничений                                      │
```

```
│                                                                    │
└────────────────────────────────────────────────────────────────────┘
```

1.4. Интеграция AI с Workflow Automation

1.4.1. Паттерны интеграции AI в workflows

```
┌──────────────────────────────────────────────────────────────────────┐
│                    ПАТТЕРНЫ ИНТЕГРАЦИИ AI В WORKFLOWS                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ПАТТЕРН 1: AI AS ACTION (AI как действие)                           │
│  ═══════════════════════════════════════════                         │
│  ┌─────────┐    ┌─────────┐    ┌─────────┐    ┌─────────┐           │
│  │ Trigger │──►│ Process │──►│  AI     │──►│ Output  │           │
│  │         │    │  Data   │    │ Action  │    │         │           │
│  └─────────┘    └─────────┘    └─────────┘    └─────────┘           │
│                                                                      │
│  Пример: Заявка → Обработка → Классификация AI → Маршрутизация       │
│  Использование: Классификация, генерация контента, анализ            │
│                                                                      │
│  ПАТТЕРН 2: AI AS ROUTER (AI как маршрутизатор)                      │
│  ═══════════════════════════════════════════                         │
│                                                                      │
│                    ┌──────────┐                                      │
│                    │    AI    │                                      │
│                    │ Classifier│                                     │
│                    └──────────┘                                      │
│                         │                                            │
│              ┌──────────┼──────────┐                                 │
│              │          │          │                                 │
│              ▼          ▼          ▼                                 │
│        ┌─────────┐  ┌─────────┐  ┌─────────┐                        │
│        │Workflow │  │Workflow │  │Workflow │                        │
│        │   A     │  │   B     │  │   C     │                        │
│        └─────────┘  └─────────┘  └─────────┘                        │
│                                                                      │
│  Пример: Заявка → AI определяет тип → Разные workflows               │
│  Использование: Умная маршрутизация, приоритизация                   │
│                                                                      │
│  ПАТТЕРН 3: AI AGENT IN WORKFLOW (AI-агент в workflow)               │
│  ═══════════════════════════════════════════                         │
│  ┌─────────┐    ┌──────────────────────┐    ┌─────────┐            │
│  │ Trigger │──►│  AI Agent (Multi-step) │──►│ Output  │            │
│  │         │    │  Research → Analyze →  │    │         │            │
│  │         │    │  Write                 │    │         │            │
│  └─────────┘    └──────────────────────┘    └─────────┘            │
│                                                                      │
│  Пример: Запрос → Multi-Agent система → Отчёт                        │
│  Использование: Сложные исследовательские задачи                     │
│                                                                      │
│  ПАТТЕРН 4: HUMAN-IN-THE-LOOP с AI                                   │
│  ═══════════════════════════════════════════                         │
│  ┌─────────┐    ┌─────────┐    ┌─────────┐    ┌─────────┐           │
│  │ Trigger │──►│   AI    │──►│ Human   │──►│ Execute │           │
│  │         │    │ Process │    │ Review  │    │         │           │
│  └─────────┘    └─────────┘    └─────────┘    └─────────┘           │
│                                                                      │
│  Пример: Заявка → AI обработка → Проверка человеком → Выполнение     │
│  Использование: Критические решения, комплаенс                       │
│                                                                      │
└──────────────────────────────────────────────────────────────────────┘
```

1.4.2. Интеграция с российскими LLM API

Пример настройки HTTP-запроса к YandexGPT в n8n:

```
{
  "node": {
    "name": "YandexGPT",
    "type": "n8n-nodes-base.httpRequest",
    "parameters": {
      "method": "POST",
      "url": "https://llm.api.cloud.yandex.net/foundationModels/v1/completion",
      "headers": {
```

```
        "Content-Type": "application/json",
        "Authorization": "Bearer {{ $env.YANDEX_IAM_TOKEN }}",
        "x-folder-id": "{{ $env.YANDEX_FOLDER_ID }}"
      },
      "body": {
        "modelUri": "gpt://{{ $env.YANDEX_FOLDER_ID }}/yandexGPT/latest",
        "completionOptions": {
          "stream": false,
          "temperature": 0.7,
          "maxTokens": 1000
        },
        "messages": [
          {
            "role": "system",
            "text": "Вы — полезный ассистент."
          },
          {
            "role": "user",
            "text": "{{ $json.query }}"
          }
        ]
      }
    }
  }
}
```

1.5. Безопасность и комплаенс в workflow automation

1.5.1. Требования безопасности для РФ

```
┌──────────────────────────────────────────────────────────────────────────┐
│                    ТРЕБОВАНИЯ БЕЗОПАСНОСТИ ДЛЯ РФ                        │
├──────────────────────────────────────────────────────────────────────────┤
│                                                                          │
│  152-ФЗ О ПЕРСОНАЛЬНЫХ ДАННЫХ:                                           │
│  ────────────────────────────────────                                    │
│  • Хранение ПДн на территории РФ                                         │
│  • Согласие субъекта на обработку                                        │
│  • Шифрование при передаче                                               │
│  • Реестр обработок ПДн                                                  │
│  • Уведомление Роскомнадзора                                             │
│                                                                          │
│  ТЕХНИЧЕСКИЕ МЕРЫ:                                                       │
│  ────────────────────                                                    │
│  • HTTPS для всех соединений                                             │
│  • API keys в переменных окружения                                       │
│  • Audit logging всех действий                                           │
│  • Role-based access control (RBAC)                                      │
│  • Regular security updates                                              │
│                                                                          │
│  РЕКОМЕНДАЦИИ ДЛЯ WORKFLOW:                                              │
│  ──────────────────────────                                              │
│  • Не хранить ключи API в workflow definitions                           │
│  • Использовать credentials management                                   │
│  • Включить двухфакторную аутентификацию                                 │
│  • Регулярно ротировать ключи доступа                                    │
│  • Мониторить подозрительную активность                                  │
│                                                                          │
└──────────────────────────────────────────────────────────────────────────┘
```

1.5.2. Best practices для production workflows

Практика	Описание	Реализация
Error Handling	Обработка всех возможных ошибок	Try-catch блоки, retry logic
Logging	Полное логирование выполнения	Structured logs, correlation IDs
Rate Limiting	Ограничение частоты вызовов	Token bucket, circuit breaker
Data Validation	Проверка входных данных	Schema validation, sanitization
Secrets Management	Безопасное хранение ключей	Environment variables, vault
Version Control	Версионирование workflows	Git integration, change tracking
Testing	Тестирование перед деплоем	Unit tests, integration tests

Практика	Описание	Реализация
Monitoring	Мониторинг выполнения	Dashboards, alerts

🔬 ЛАБОРАТОРНАЯ РАБОТА №4

Тема: Создание автоматизированного workflow с интеграцией AI

📋 ПАСПОРТ ЛАБОРАТОРНОЙ РАБОТЫ

Параметр	Значение
Номер работы	4
Тема	Создание автоматизированного workflow с интеграцией AI
Время выполнения	2 часа (аудиторно) + 5 часов (самостоятельно)
Формат сдачи	Pull Request в ветку `week4` основного репозитория
Пререквизиты	Успешное выполнение лабораторных работ №1-3
Ресурсы	Все сервисы доступны с территории РФ

🎯 ЦЕЛИ И ЗАДАЧИ

Цели работы

1. ✅ Понимать архитектуру платформ workflow automation
2. ✅ Развернуть n8n или использовать ApiX-Drive
3. ✅ Создать workflow с интеграцией российского LLM API
4. ✅ Реализовать ветвление логики на основе AI-классификации
5. ✅ Адаптировать workflow под тематику дипломной работы

Задачи

№	Задача	Результат
1	Развернуть платформу автоматизации	Работоспособный n8n/ApiX-Drive
2	Создать базовый workflow	Есть базовый workflow
3	Интегрировать YandexGPT/GigaChat	Интегрированные AI-узлы
4	Реализовать условную логику	Workflow с логикой ветвления
5	Адаптировать под специальность	Тематико-специфичный workflow
6	Протестировать и задокументировать	Тесты + документация

📁 ТРЕБОВАНИЯ К СТРУКТУРЕ ПРОЕКТА

Обновление репозитория

Создать ветку `week4` в существующем репозитории:

```
# Переход в репозиторий
cd ~/ai-course-labs

# Создание ветки week4 от main
git checkout main
git pull origin main
git checkout -b week4

# Создание структуры для Week 4
mkdir -p week4/workflows
mkdir -p week4/src
mkdir -p week4/docker
mkdir -p week4/notebooks
```

```
mkdir -p week4/tests
mkdir -p week4/docs
```

Итоговая структура week4/

```
week4/
├── .env.example                    # Шаблон переменных окружения
├── requirements.txt                # Зависимости Python
├── docker/
│   ├── docker-compose.yml          # Конфигурация для n8n
│   └── .env                        # Переменные для Docker
├── workflows/
│   ├── basic_workflow.json         # Базовый workflow (export)
│   ├── ai_classifier.json          # Workflow с AI-классификацией
│   └── specialty_workflow.json     # Адаптированный workflow
├── src/
│   ├── __init__.py
│   ├── webhook_handler.py          # Обработчик webhook
│   └── api_client.py               # Клиент для API
├── notebooks/
│   └── lab4_workflow_demo.ipynb    # Демонстрация работы
├── tests/
│   └── test_workflow.py            # Тесты workflow
└── docs/
    └── report.md                   # Отчёт по работе
```

🔧 ЧАСТЬ 1: НАСТРОЙКА ОКРУЖЕНИЯ

Шаг 1.1: Выбор платформы

Вариант А: n8n (self-hosted) — рекомендуется

Преимущества:

- ✅ Бесплатно для self-hosted
- ✅ Полный контроль над данными
- ✅ Соответствие 152-ФЗ
- ✅ Гибкая кастомизация

Вариант Б: ApiX-Drive (российская платформа)

Преимущества:

- ✅ Не требует развёртывания
- ✅ Российская поддержка
- ✅ Готовые интеграции с РФ сервисами
- ✅ Простой интерфейс

Шаг 1.2: Развёртывание n8n через Docker

Создайте файл `week4/docker/docker-compose.yml`:

```yaml
# docker-compose.yml
# Конфигурация для развёртывания n8n

version: '3.8'

services:
  n8n:
    image: n8nio/n8n:latest
    container_name: n8n_workflow
    restart: unless-stopped
    ports:
      - "5678:5678"
    environment:
      - N8N_HOST=${N8N_HOST:-localhost}
      - N8N_PORT=5678
      - N8N_PROTOCOL=http
      - NODE_ENV=production
      - WEBHOOK_URL=${WEBHOOK_URL:-http://localhost:5678/}
      - GENERIC_TIMEZONE=${GENERIC_TIMEZONE:-Europe/Moscow}
      - N8N_ENCRYPTION_KEY=${N8N_ENCRYPTION_KEY:-change-this-key}
```

```yaml
      # Yandex Cloud credentials
      - YANDEX_IAM_TOKEN=${YANDEX_IAM_TOKEN}
      - YANDEX_FOLDER_ID=${YANDEX_FOLDER_ID}
      # GigaChat credentials (альтернатива)
      - GIGACHAT_CLIENT_ID=${GIGACHAT_CLIENT_ID}
      - GIGACHAT_CLIENT_SECRET=${GIGACHAT_CLIENT_SECRET}
    volumes:
      - n8n_data:/home/node/.n8n
      - ./workflows:/home/node/workflows
    networks:
      - n8n_network

  postgres:
    image: postgres:15-alpine
    container_name: n8n_postgres
    restart: unless-stopped
    environment:
      - POSTGRES_USER=${POSTGRES_USER:-n8n}
      - POSTGRES_PASSWORD=${POSTGRES_PASSWORD:-n8n}
      - POSTGRES_DB=${POSTGRES_DB:-n8n}
    volumes:
      - postgres_data:/var/lib/postgresql/data
    networks:
      - n8n_network
    healthcheck:
      test: ["CMD-SHELL", "pg_isready -U n8n"]
      interval: 10s
      timeout: 5s
      retries: 5

volumes:
  n8n_data:
    driver: local
  postgres_data:
    driver: local

networks:
  n8n_network:
    driver: bridge
```

Создайте файл `week4/docker/.env`:

```env
# .env
# Переменные окружения для n8n

# N8N Configuration
N8N_HOST=localhost
WEBHOOK_URL=http://localhost:5678/
N8N_ENCRYPTION_KEY=your-secure-encryption-key-change-this

# Timezone
GENERIC_TIMEZONE=Europe/Moscow

# Database
POSTGRES_USER=n8n
POSTGRES_PASSWORD=n8n_secure_password_change_this
POSTGRES_DB=n8n

# Yandex Cloud API
YANDEX_IAM_TOKEN=your_iam_token_here
YANDEX_FOLDER_ID=your_folder_id_here

# GigaChat API (альтернатива)
GIGACHAT_CLIENT_ID=your_client_id_here
GIGACHAT_CLIENT_SECRET=your_client_secret_here

# Webhook Secret (для безопасности)
WEBHOOK_SECRET=your_webhook_secret_here
```

Шаг 1.3: Запуск n8n

```bash
# Переход в директорию с docker-compose
cd week4/docker

# Запуск контейнеров
docker-compose up -d
```

```
# Проверка статуса
docker-compose ps

# Просмотр логов
docker-compose logs -f n8n

# Остановка
docker-compose down
```

Доступ к интерфейсу:

- URL: http://localhost:5678
- Первый вход: создание учётной записи администратора

💻 ЧАСТЬ 2: СОЗДАНИЕ БАЗОВОГО WORKFLOW

Шаг 2.1: Базовый workflow (обработка заявок)

Сценарий: Автоматическая обработка входящих заявок с классификацией через AI

```
┌──────────────────────────────────────────────────────────────────────┐
│                    БАЗОВЫЙ WORKFLOW: ОБРАБОТКА ЗАЯВОК                 │
├──────────────────────────────────────────────────────────────────────┤
│                                                                      │
│  ┌──────────────┐    ┌──────────────┐    ┌──────────────┐    ┌──────────────┐  │
│  │   Webhook    │──►│  YandexGPT   │──►│   IF/Else    │──►│   Google     │  │
│  │   (Trigger)  │    │ (Classifier) │    │  (Branch)    │    │   Sheets     │  │
│  └──────────────┘    └──────────────┘    └──────────────┘    └──────────────┘  │
│       │                  │                  │                  │          │
│       │                  │                  │                  │          │
│       ▼                  ▼                  ▼                  ▼          │
│   Новая заявка      Классификация      Ветвление по       Сохранение       │
│   с сайта           типа заявки        типу заявки        в таблицу        │
│                                                                           │
│       │                  │                  │                            │
│       │                  │                  │                            │
│       └──────────────────┴──────────────────┴────────────────────────────┘
│                                  │                                        │
│                                  ▼                                        │
│                            ┌──────────────┐                               │
│                            │  Telegram    │                               │
│                            │  Notification│                               │
│                            └──────────────┘                               │
│                                  │                                        │
│                                  ▼                                        │
│                            Уведомление                                    │
│                            менеджеру                                      │
│                                                                           │
└──────────────────────────────────────────────────────────────────────────┘
```

Шаг 2.2: Экспорт workflow в JSON

Создайте файл `week4/workflows/basic_workflow.json`:

```
{
  "name": "Обработка заявок с AI-классификацией",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "application",
        "responseMode": "lastNode",
        "options": {}
      },
      "name": "Webhook",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 1,
      "position": [250, 300],
      "webhookId": "unique-webhook-id"
    },
    {
      "parameters": {
```

```
        "method": "POST",
        "url": "https://llm.api.cloud.yandex.net/foundationModels/v1/completion",
        "authentication": "genericCredentialType",
        "genericAuthType": "httpHeaderAuth",
        "sendHeaders": true,
        "headerParameters": {
          "parameters": [
            {
              "name": "Content-Type",
              "value": "application/json"
            },
            {
              "name": "Authorization",
              "value": "Bearer {{ $env.YANDEX_IAM_TOKEN }}"
            },
            {
              "name": "x-folder-id",
              "value": "{{ $env.YANDEX_FOLDER_ID }}"
            }
          ]
        },
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "modelUri",
              "value": "gpt://{{ $env.YANDEX_FOLDER_ID }}/yandexGPT/latest"
            },
            {
              "name": "completionOptions.temperature",
              "value": "0.3"
            },
            {
              "name": "completionOptions.maxTokens",
              "value": "100"
            },
            {
              "name": "messages",
              "value": "=[{\"role\":\"system\",\"text\":\"Классифицируй заявку как одну из категорий: техническая_поддержка, продажа,
вопрос, жалоба. Ответь только названием категории.\"},{\"role\":\"user\",\"text\":\"{{ $json.body.message }}\"}]"
            }
          ]
        },
        "options": {}
      },
      "name": "YandexGPT Classifier",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
      "position": [450, 300]
    },
    {
      "parameters": {
        "conditions": {
          "string": [
            {
              "value1": "{{ $json.result.alternatives[0].message.text }}",
              "operation": "equals",
              "value2": "техническая_поддержка"
            }
          ]
        }
      },
      "name": "IF Tech Support",
      "type": "n8n-nodes-base.if",
      "typeVersion": 1,
      "position": [650, 300]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://api.telegram.org/bot{{ $env.TELEGRAM_BOT_TOKEN }}/sendMessage",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "chat_id",
              "value": "{{ $env.TELEGRAM_CHAT_ID }}"
```

```
            },
            {
              "name": "text",
              "value": "🔧 Новая заявка в техподдержку:\n{{ $json.body.message }}\n\nКонтакт: {{ $json.body.contact }}"
            }
          ]
        }
      },
      "name": "Telegram Notification",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
      "position": [850, 300]
    }
  ],
  "connections": {
    "Webhook": {
      "main": [
        [
          {
            "node": "YandexGPT Classifier",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "YandexGPT Classifier": {
      "main": [
        [
          {
            "node": "IF Tech Support",
            "type": "main",
            "index": 0
          }
        ]
      ]
    },
    "IF Tech Support": {
      "main": [
        [
          {
            "node": "Telegram Notification",
            "type": "main",
            "index": 0
          }
        ]
      ]
    }
  },
  "settings": {
    "executionOrder": "v1"
  },
  "staticData": null,
  "tags": []
}
```

🎓 ЧАСТЬ 3: АДАПТАЦИЯ ПОД СПЕЦИАЛЬНОСТЬ

Задание 3.1: Создание специализированного workflow

Инструкция: Создайте workflow, релевантный вашей научной специальности. Используйте шаблон ниже.

Таблица вариантов адаптации

Специальность	Сценарий workflow	Триггер	AI-действие	Результат
Информационная безопасность	Анализ логов безопасности	Webhook от SIEM	Классификация инцидентов	Уведомление SOC
Автоматизация процессов	Мониторинг оборудования	Polling PLC	Прогноз неисправностей	Заявка на ТО
Строительство	Контроль документации	File upload	Проверка соответствия СНиП	Отчёт о проверке
Энергетика	Прогноз потребления	Schedule	Анализ временных рядов	Рекомендации по оптимизации
Транспорт	Оптимизация маршрутов	GPS data	Анализ трафика	Обновлённый маршрут
Программная инженерия	Code review	Git webhook	Анализ кода	Отчёт о качестве

Специальность	Сценарий workflow	Триггер	AI-действие	Результат
Медицинская техника	Мониторинг пациентов	Device data	Анализ показателей	Алерт врачу

Листинг 3.1: Шаблон специализированного workflow

```
{
  "name": "Специализированный workflow для [СПЕЦИАЛЬНОСТЬ]",
  "description": "[Описание workflow и его назначения]",
  "author": "[ВАША ФАМИЛИЯ]",
  "diploma_topic": "[ТЕМА ДИПЛОМНОЙ РАБОТЫ]",
  "nodes": [
    {
      "parameters": {
        "httpMethod": "POST",
        "path": "specialty-webhook",
        "responseMode": "lastNode"
      },
      "name": "Trigger",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 1,
      "position": [250, 300]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://llm.api.cloud.yandex.net/foundationModels/v1/completion",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "modelUri",
              "value": "gpt://{{ $env.YANDEX_FOLDER_ID }}/yandexGPT/latest"
            },
            {
              "name": "messages",
              "value": "=[{\"role\":\"system\",\"text\":\"[ВАШ ПРОМПТ ДЛЯ СПЕЦИАЛЬНОСТИ]\"},{\"role\":\"user\",\"text\":\"{{
$json.body.data }}\"}]"
            }
          ]
        }
      },
      "name": "AI Processor",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
      "position": [450, 300]
    },
    {
      "parameters": {},
      "name": "Output Action",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
      "position": [650, 300]
    }
  ],
  "connections": {
    "Trigger": {
      "main": [[{"node": "AI Processor", "type": "main", "index": 0}]]
    },
    "AI Processor": {
      "main": [[{"node": "Output Action", "type": "main", "index": 0}]]
    }
  }
}
```

Листинг 3.2: Пример для информационной безопасности

```
{
  "name": "Анализ инцидентов безопасности",
  "description": "Автоматическая классификация и маршрутизация инцидентов ИБ",
  "author": "[Ваша фамилия]",
  "specialty": "Информационная безопасность",
  "nodes": [
    {
      "parameters": {
```

```
        "httpMethod": "POST",
        "path": "security-incident",
        "responseMode": "lastNode"
      },
      "name": "SIEM Webhook",
      "type": "n8n-nodes-base.webhook",
      "typeVersion": 1,
      "position": [250, 300]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://llm.api.cloud.yandex.net/foundationModels/v1/completion",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "modelUri",
              "value": "gpt://{{ $env.YANDEX_FOLDER_ID }}/yandexGPT/latest"
            },
            {
              "name": "completionOptions.temperature",
              "value": "0.2"
            },
            {
              "name": "messages",
              "value": "=[{\"role\":\"system\",\"text\":\"Классифицируй инцидент безопасности по уровню критичности: critical, high,
medium, low. Также определи тип атаки. Ответь в формате JSON: {\\\"level\\\": \\\"...\\\", \\\"type\\\": \\\"...\\\",
\\\"recommendation\\\": \\\"...\\\"}\"},{\"role\":\"user\",\"text\":\"{{ $json.body.log_entry }}\"}]"
            }
          ]
        }
      },
      "name": "Security AI Classifier",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
      "position": [450, 300]
    },
    {
      "parameters": {
        "conditions": {
          "string": [
            {
              "value1": "{{ JSON.parse($json.result.alternatives[0].message.text).level }}",
              "operation": "equals",
              "value2": "critical"
            }
          ]
        }
      },
      "name": "IF Critical",
      "type": "n8n-nodes-base.if",
      "typeVersion": 1,
      "position": [650, 300]
    },
    {
      "parameters": {
        "method": "POST",
        "url": "https://api.telegram.org/bot{{ $env.TELEGRAM_BOT_TOKEN }}/sendMessage",
        "sendBody": true,
        "bodyParameters": {
          "parameters": [
            {
              "name": "chat_id",
              "value": "{{ $env.SOC_CHAT_ID }}"
            },
            {
              "name": "text",
              "value": "🚨 КРИТИЧЕСКИЙ ИНЦИДЕНТ ИБ\n\nТип: {{ JSON.parse($json.result.alternatives[0].message.text).type
}}\n\nРекомендация: {{ JSON.parse($json.result.alternatives[0].message.text).recommendation }}\n\nЛог: {{ $json.body.log_entry }}"
            }
          ]
        }
      },
      "name": "SOC Alert",
      "type": "n8n-nodes-base.httpRequest",
      "typeVersion": 4,
```

```
      "position": [850, 300]
    }
  ],
  "connections": {
    "SIEM Webhook": {
      "main": [[{"node": "Security AI Classifier", "type": "main", "index": 0}]]
    },
    "Security AI Classifier": {
      "main": [[{"node": "IF Critical", "type": "main", "index": 0}]]
    },
    "IF Critical": {
      "main": [[{"node": "SOC Alert", "type": "main", "index": 0}]]
    }
  }
}
```

🐍 ЧАСТЬ 4: PYTHON-ИНТЕГРАЦИЯ С WORKFLOW

Шаг 4.1: Webhook handler для тестирования

Создайте файл `week4/src/webhook_handler.py` :

```python
# -*- coding: utf-8 -*-
"""
Обработчик webhook для тестирования workflow
Лабораторная работа №4
Дисциплина: Искусственный интеллект

Автор: [ФИО]
Группа: [НОМЕР ГРУППЫ]
Дата: 2026
"""

import os
import json
import requests
import logging
from typing import Dict, Any, Optional
from datetime import datetime
from dotenv import load_dotenv

# Настройка логирования
logging.basicConfig(
    level=logging.INFO,
    format='%(asctime)s - %(name)s - %(levelname)s - %(message)s'
)
logger = logging.getLogger(__name__)

# Загрузка переменных окружения
load_dotenv()

class WorkflowClient:
    """
    Клиент для взаимодействия с n8n workflow.

    Атрибуты:
        base_url: URL n8n сервера
        webhook_path: Путь webhook
        secret: Секретный ключ для аутентификации
    """

    def __init__(
        self,
        base_url: str = "http://localhost:5678",
        webhook_path: str = "application",
        secret: Optional[str] = None
    ):
        self.base_url = base_url.rstrip('/')
        self.webhook_path = webhook_path
        self.secret = secret or os.getenv("WEBHOOK_SECRET")
        self.webhook_url = f"{self.base_url}/webhook/{webhook_path}"

        logger.info(f"WorkflowClient инициализирован: {self.webhook_url}")
```

```python
    def send_application(
        self,
        message: str,
        contact: str,
        priority: str = "normal"
    ) -> Dict[str, Any]:
        """
        Отправка заявки в workflow.

        Args:
            message: Текст заявки
            contact: Контактная информация
            priority: Приоритет (low, normal, high)

        Returns:
            Dict: Ответ от workflow
        """
        payload = {
            "message": message,
            "contact": contact,
            "priority": priority,
            "timestamp": datetime.now().isoformat()
        }

        headers = {
            "Content-Type": "application/json"
        }

        if self.secret:
            headers["X-Webhook-Secret"] = self.secret

        logger.info(f"Отправка заявки: {message[:50]}...")

        try:
            response = requests.post(
                self.webhook_url,
                headers=headers,
                json=payload,
                timeout=30
            )
            response.raise_for_status()

            result = response.json()

            logger.info(f"Workflow выполнил обработку")

            return {
                "success": True,
                "response": result,
                "status_code": response.status_code
            }

        except requests.exceptions.RequestException as e:
            logger.error(f"Ошибка отправки: {e}")
            return {
                "success": False,
                "error": str(e),
                "status_code": None
            }

    def send_security_incident(
        self,
        log_entry: str,
        source_ip: str,
        event_type: str
    ) -> Dict[str, Any]:
        """
        Отправка инцидента безопасности (для ИБ-специальности).

        Args:
            log_entry: Лог события
            source_ip: IP-адрес источника
            event_type: Тип события

        Returns:
            Dict: Ответ от workflow
        """
```

```python
        payload = {
            "log_entry": log_entry,
            "source_ip": source_ip,
            "event_type": event_type,
            "timestamp": datetime.now().isoformat()
        }

        headers = {
            "Content-Type": "application/json"
        }

        if self.secret:
            headers["X-Webhook-Secret"] = self.secret

        webhook_url = f"{self.base_url}/webhook/security-incident"

        logger.info(f"Отправка инцидента ИБ: {event_type}")

        try:
            response = requests.post(
                webhook_url,
                headers=headers,
                json=payload,
                timeout=30
            )
            response.raise_for_status()

            result = response.json()

            logger.info(f"Инцидент обработан")

            return {
                "success": True,
                "response": result,
                "status_code": response.status_code
            }

        except requests.exceptions.RequestException as e:
            logger.error(f"Ошибка отправки: {e}")
            return {
                "success": False,
                "error": str(e),
                "status_code": None
            }

    def check_workflow_status(self) -> Dict[str, Any]:
        """Проверка доступности n8n."""
        try:
            response = requests.get(
                f"{self.base_url}/healthz",
                timeout=5
            )
            return {
                "available": response.status_code == 200,
                "status_code": response.status_code
            }
        except Exception as e:
            return {
                "available": False,
                "error": str(e)
            }


# Точка входа для тестирования
if __name__ == "__main__":
    print("=" * 80)
    print("ЛАБОРАТОРНАЯ РАБОТА №4")
    print("Тестирование workflow automation")
    print("=" * 80)

    client = WorkflowClient()

    # Проверка доступности
    print("\nПроверка доступности n8n...")
    status = client.check_workflow_status()

    if status.get("available"):
```

```
        print("✅ n8n доступен")
    else:
        print(f"❌ n8n недоступен: {status.get('error')}")
        exit(1)

    # Тестовая заявка
    print("\n" + "=" * 80)
    print("ТЕСТОВАЯ ЗАЯВКА")
    print("=" * 80)

    result = client.send_application(
        message="Не работает вход в систему, ошибка 403",
        contact="user@example.com",
        priority="high"
    )

    if result["success"]:
        print("✅ Заявка отправлена успешно")
        print(f"Статус код: {result['status_code']}")
    else:
        print(f"❌ Ошибка: {result['error']}")

    print("=" * 80)
```

📝 ЧАСТЬ 5: ТРЕБОВАНИЯ К ОТЧЁТУ

Структура отчёта (week4/docs/report.md)

```
# Отчёт по лабораторной работе №4
## Дисциплина: Искусственный интеллект

---

## Общая информация

| Параметр | Значение |
|----------|----------|
| **Студент** | [ФИО полностью] |
| **Группа** | [Номер группы] |
| **Дата выполнения** | [ДД.ММ.ГГГГ] |
| **Специальность** | [Название специальности] |
| **Тема диплома** | [Тема дипломной работы] |

---

## 1. Цель работы

[Опишите цель работы своими словами]

---

## 2. Выполненные задачи

- [ ] Развёрнута платформа автоматизации (n8n/ApiX-Drive)
- [ ] Создан базовый workflow
- [ ] Интегрирован российский LLM API
- [ ] Реализовано ветвление логики
- [ ] Создан специализированный workflow
- [ ] Код загружен в GitHub

---

## 3. Ход работы

### 3.1. Настройка платформы

[Скриншот интерфейса n8n/ApiX-Drive]
[Описание конфигурации]

### 3.2. Базовый workflow

[Схема workflow]
[Описание узлов и связей]
```

```
### 3.3. Интеграция с YandexGPT/GigaChat

[Конфигурация AI-ноды]
[Пример промпта]

### 3.4. Специализированный workflow

[Описание адаптированного workflow]
[Схема для вашей специальности]
[Листинг JSON]

### 3.5. Тестирование

[Примеры запросов и ответов]
[Скриншоты выполнения]
[Логи работы]

### 3.6. Интеграция с дипломом

[Описание использования в дипломной работе]
[Как workflow автоматизирует процессы диплома]

---

## 4. Результаты

| Критерий | Статус |
|----------|--------|
| Платформа развёрнута | ✅ |
| Workflow работает | ✅ |
| AI интегрирован | ✅ |
| Специализация выполнена | ✅ |
| Код в GitHub | ✅ |

---

## 5. Выводы

[Что изучили, трудности, планы]

---

## 6. Список источников

. . .
```

📊 КРИТЕРИИ ОЦЕНИВАНИЯ

Критерий	Баллы	Требования
Развёртывание платформы	20	n8n/ApiX-Drive работает
Базовый workflow	20	Workflow выполняется успешно
AI-интеграция	20	YandexGPT/GigaChat подключён
Специализация под диплом	25	Workflow релевантен специальности
Ветвление логики	10	IF/Else реализовано
GitHub (ветка week4)	5	Код загружен, структура соблюдена
ИТОГО	**100**	

🚀 ЧАСТЬ 6: ЗАГРУЗКА В GITHUB

```
# Переход в директорию репозитория
cd ~/ai-course-labs

# Проверка ветки
git branch  # Должно быть * week4

# Добавление файлов
```

```
git add week4/

# Коммит
git commit -m "Lab 4: Workflow Automation with AI integration

- Deployed n8n platform via Docker
- Created basic workflow with webhook trigger
- Integrated YandexGPT for AI classification
- Implemented branching logic based on AI output
- Created specialty-specific workflow for diploma
- Added Python webhook handler for testing

Author: [ВАША ФАМИЛИЯ]
Group: [НОМЕР ГРУППЫ]"

# Отправка в GitHub
git push -u origin week4
```